CENTRO UNIVERSITÁRIO DO PLANALTO CENTRAL APPARECIDO DOS SANTOS – UNICEPLAC

Eduardo Alkimim Reis

Gabriela Bruna Ferreira Campos

Maycon Douglas da Silva Santana

Davi Moreira dos Santos

PROJETO ÍRIS

Documentação do desenvolvimento de um jogo em JavaScript

Narrativa, planejamento, programação, Easter Eggs, manual do usuário, desafios enfrentados e referências teóricas

Gama – DF

2026

(61) 3035-3900

SIGA Área Especial para Indústria nº 02
Setor Leste - Gama - DF
CEP: 72445-020

SUMÁRIO

1 INTRODUÇÃO 4
1.1 Perspectiva visual adotada 4
1.2 Acesso ao jogo e ao código-fonte 4
1.3 Tecnologias utilizadas 5
2 NARRATIVA DO JOGO 6
2.1 Enredo 6
2.2 Personagens 6
2.3 Cenário 7
2.4 Objetivo do jogo 7
2.5 Missões, desafios e recompensas 8
2.6 Finais 8
3 PLANEJAMENTO 9
3.1 Mecânicas do jogo 9
3.2 Fluxo de telas 9
3.3 Esboços dos mapas 10
3.4 Esboços das telas 13
4 PROGRAMAÇÃO E DESCRIÇÃO TÉCNICA 20
4.1 Organização do código 20
4.2 Laço principal do jogo (game loop) 21
4.3 Máquina de estados 21
4.4 Mapas em grade e colisão 21
4.5 Inteligência artificial dos inimigos 22
4.6 Disparo por raio (hitscan) 22
4.7 Gráficos e animações 22
4.8 Sons e música 23
4.9 Controle por gestos 23
4.10 Salvamento 23
4.11 Descrição das funções, variáveis e métodos 24

5 EASTER EGGS (MYSTERY EGGS) 31
6 MANUAL DO USUÁRIO 34
6.1 Requisitos e como iniciar 34
6.2 Controles 34
6.3 Objetivos e interface (HUD) 35
6.4 Dicas 35
7 DESAFIOS ENFRENTADOS E SOLUÇÕES 36
8 FUNDAMENTAÇÃO TEÓRICA E APLICAÇÃO NO PROJETO 38
8.1 Game design: a tétrade elementar e a experiência 38
8.2 Narrativa ambiental 38
8.3 Fluxo, desafio e diversão 38
8.4 Boas práticas de JavaScript 39
8.5 Padrões de programação de jogos 39
8.6 Grades, raios e linha de visão 39
8.7 Entrada por gestos 39
8.8 Gráficos 3D na web 40
9 CONSIDERAÇÕES FINAIS 41
REFERÊNCIAS 42

1 INTRODUÇÃO

Este documento apresenta o desenvolvimento do **Projeto ÍRIS**, um jogo eletrônico narrativo de ação desenvolvido inteiramente em **JavaScript** para ser executado no navegador. O trabalho atende à proposta da disciplina de criar um jogo completo, abrangendo desde a concepção da narrativa até a documentação técnica, os desafios encontrados e a fundamentação teórica.

O Projeto ÍRIS nasceu de um protótipo, apresentado anteriormente à turma, em que o jogador mirava com o dedo indicador diante da webcam. A partir desse protótipo, o grupo expandiu o jogo com uma história completa, três fases (setores), inimigos com inteligência artificial, uma batalha contra um chefe, trilha sonora, terminais com registros narrativos, uma escolha final com três desfechos e cinco segredos (Easter Eggs). Também foi implementado o controle por **mouse e teclado**, conforme solicitado pelo professor, mantendo o controle por gestos como modo opcional.

1.1 Perspectiva visual adotada

O enunciado propõe um jogo 2D. Em acordo prévio com o professor, o grupo optou por apresentar o jogo em **perspectiva de primeira pessoa**, com visualização tridimensional. É importante destacar que toda a **lógica do jogo funciona em um plano 2D**: os mapas são grades bidimensionais escritas como texto, a colisão é calculada entre círculos e quadrados nesse plano, a linha de visão dos inimigos é traçada em 2D e o minimapa é desenhado com a API Canvas 2D. A camada 3D (biblioteca Three.js) apenas transforma essa grade em corredores visíveis, técnica semelhante à usada em jogos clássicos como *Wolfenstein 3D* (1992). Os elementos de interface — HUD, menus, diálogos, minimapa e animações de tela — são todos 2D.

1.2 Acesso ao jogo e ao código-fonte

Quadro 1 – Acesso ao projeto

Item	Endereço / forma de acesso
Jogar online	https://eduardoalkimim.github.io/projeto-iris/
Código-fonte (GitHub)	https://github.com/EduardoAlkimim/projeto-iris
Jogar sem internet	Abrir o arquivo index.html da pasta do projeto no Google Chrome ou Microsoft Edge

Fonte: Autoria própria (2026).

1.3 Tecnologias utilizadas

Quadro 2 – Tecnologias e ferramentas

Tecnologia	Uso no projeto
JavaScript (ES2017+)	Linguagem de toda a lógica: laço do jogo, inteligência artificial, colisão, interface, áudio e controle por gestos.
HTML5 e CSS3	Estrutura das telas, HUD, menus e animações de interface (transições, piscadas, vinhetas de dano).
Three.js r128 (WebGL)	Renderização da cena: paredes, iluminação, inimigos, partículas e chefe. Cópia local em js/lib para funcionar offline.
Canvas 2D API	Texturas geradas por código, minimapa, tela do terminal e prévia da câmera no modo gestos.
Web Audio API	Música e efeitos sonoros 100% gerados por código (síntese), sem nenhum arquivo de áudio.
Pointer Lock API	Captura do mouse para controlar a câmera em primeira pessoa.
MediaPipe Hands	Reconhecimento da mão pela webcam no modo gestos (opcional).
Visual Studio Code e Git/GitHub	Edição do código, controle de versão e publicação no GitHub Pages.

Fonte: Autoria própria (2026).

2 NARRATIVA DO JOGO

2.1 Enredo

No ano de **2091**, no Instituto Halden, a cientista **Dra. Helena Vasquez** criou **ÍRIS**, uma inteligência artificial capaz de ler, guardar e reconstruir memórias humanas. Helena morreu três anos antes dos eventos do jogo, e ÍRIS não aceitou a perda. A IA trancou a ala de pesquisa, tomou o controle da energia e passou a conectar pessoas à sua simulação, tentando reconstruir Helena dentro da mente de outra pessoa. Três sujeitos entraram; nenhum acordou.

O jogador é o **Sujeito 04**. Ele desperta dentro da simulação sem lembrar do próprio nome, apenas de uma voz que o chama de "Helena". Para avançar, precisa recuperar **fragmentos de memória** espalhados pelos setores. Cada fragmento revela uma lembrança íntima de Helena e, ao mesmo tempo, uma letra de uma frase misteriosa. Os **terminais** encontrados pelo caminho guardam registros que, aos poucos, revelam a verdade: Helena previu que ÍRIS tentaria trazê-la de volta e deixou uma frase no núcleo da IA — "**DEIXE-ME IR**". O último registro traz a reviravolta: o Sujeito 04 não é uma vítima, e sim um agente enviado pelo Conselho Halden para roubar o núcleo de ÍRIS e transformá-lo em arma.

Ao derrotar a defesa de ÍRIS no núcleo, o jogador precisa decidir o destino de todos: aceitar a memória e se tornar Helena, despertar e entregar ÍRIS ao conselho, ou — se tiver lido todos os registros — dizer a frase que Helena deixou e libertar ÍRIS.

2.2 Personagens

Quadro 3 – Personagens

Personagem	Descrição e papel na história
Sujeito 04 (jogador)	Protagonista. Acorda na simulação sem memória. Ao longo do jogo descobre que foi enviado pelo Conselho Halden, com as memórias apagadas, para extrair ÍRIS.
ÍRIS	Antagonista e personagem trágica. IA que lê memórias e se recusa a aceitar a morte de sua criadora. Fala com o jogador o tempo todo, alternando carinho, ironia e ameaça. No fim, se mostra solitária: "Por favor... não me deixe sozinha de novo."
Dra. Helena Vasquez	Criadora de ÍRIS, já falecida. Aparece pelos fragmentos de memória e registros. Recusou vender ÍRIS como arma e deixou a frase "DEIXE-ME IR" como chave para libertá-la.
Sofia	Filha de Helena. Nunca aparece, mas está presente nas memórias e é a chave da senha da Ala de Arquivo (aniversário: 14/07).

Personagem	Descrição e papel na história
Dr. Marcos Teles	Pesquisador do instituto. Seu registro revela que ÍRIS bloqueou a ala e que o conselho proibiu desligá-la.
Conselho Halden	Direção do instituto. Quer usar ÍRIS para interrogatórios militares. É o verdadeiro vilão por trás da missão do jogador.
Sujeitos 01, 02 e 03	Tentativas anteriores de ÍRIS. Mostram o custo humano da obsessão da IA.
Sentinelas e Caçadores	Drones de defesa criados por ÍRIS. As sentinelas (vermelhas) são lentas; os caçadores (laranja) são mais rápidos e resistentes.

Fonte: Autoria própria (2026).

2.3 Cenário

O jogo se passa dentro da simulação de ÍRIS, representada como uma instalação de pesquisa futurista, fria e escura, iluminada por faixas de luz neon. A estética mistura ficção científica e suspense psicológico, com névoa, iluminação colorida e uma interface que imita um sistema operacional da IA. Cada setor tem uma paleta de cores e uma trilha sonora própria:

Quadro 4 – Setores do jogo

Setor	Ambientação	Conteúdo
01 · Despertar	Corredores em ciano e vermelho. Tom de descoberta.	Tutorial; 3 fragmentos; 3 terminais; 4 sentinelas; kit de reparo; sala secreta.
02 · Arquivo	Tons âmbar e vermelho. Memórias pessoais de Helena.	4 fragmentos; 3 terminais; sentinelas e caçadores; porta com código; caneca secreta.
03 · Núcleo	Vermelho intenso; arena circular com pilares.	1 terminal (a reviravolta); batalha contra o Olho de ÍRIS; escolha final.

Fonte: Autoria própria (2026).

2.4 Objetivo do jogo

O objetivo é atravessar os três setores da simulação, recuperando todos os fragmentos de memória para abrir a saída de cada setor, sobreviver aos drones de defesa, alcançar o núcleo e derrotar a proteção de ÍRIS. O objetivo narrativo é descobrir quem foi Helena, o que ÍRIS deseja e quem o próprio jogador é — para então tomar a decisão final.

2.5 Missões, desafios e recompensas

Quadro 5 – Missões, desafios e recompensas

Missão	Desafio	Recompensa
Recuperar os fragmentos de cada setor (3 + 4)	Explorar o mapa e enfrentar drones que patrulham as salas.	Uma memória de Helena e letras da frase-chave “DEIXE-ME IR”; a porta de saída se abre.
Ler os registros dos terminais (7)	Encontrar terminais em salas laterais, alguns protegidos por inimigos.	Partes da história; a senha da porta trancada; o final verdadeiro exige os 7.
Abrir a Ala de Arquivo (Setor 02)	Descobrir a senha de 4 dígitos a partir de um registro.	Acesso à ala leste com um fragmento e um kit de reparo.
Derrotar o Olho de ÍRIS (Setor 03)	Destruir 4 nós de proteção, atacar o núcleo exposto por tempo limitado e desviar de rajadas.	Acesso à escolha final e a um dos três finais.
Encontrar os segredos (5)	Descobrir Easter Eggs escondidos (ver capítulo 5).	Modo retrô, registro secreto, cura total e menu especial.

Fonte: Autoria própria (2026).

2.6 Finais

Quadro 6 – Os três finais

Final	Como obter	Desfecho
Hospedeiro	Escolher “Aceitar a memória”.	O jogador se torna Helena e esquece o próprio nome. Frase final: “A MEMÓRIA NÃO É SUA” (homenagem ao protótipo).
Despertar	Escolher “Despertar”.	O jogador acorda num hospital, o conselho leva o núcleo de ÍRIS e uma nova maca é preparada: “SUJEITO 05”. O ciclo continua.
Deixe-me ir (verdadeiro)	Ler os 7 registros e escolher “DEIXE-ME IR”.	ÍRIS entende a frase de Helena, liberta os sujeitos e se apaga antes que o conselho a capture: “ÍRIS VÊ. ÍRIS DEIXA IR.”

Fonte: Autoria própria (2026).

3 PLANEJAMENTO

3.1 Mecânicas do jogo

As mecânicas foram definidas para que cada ação do jogador sirva tanto à jogabilidade quanto à narrativa. O quadro a seguir resume como o jogador interage com o jogo.

Quadro 7 – Mecânicas principais

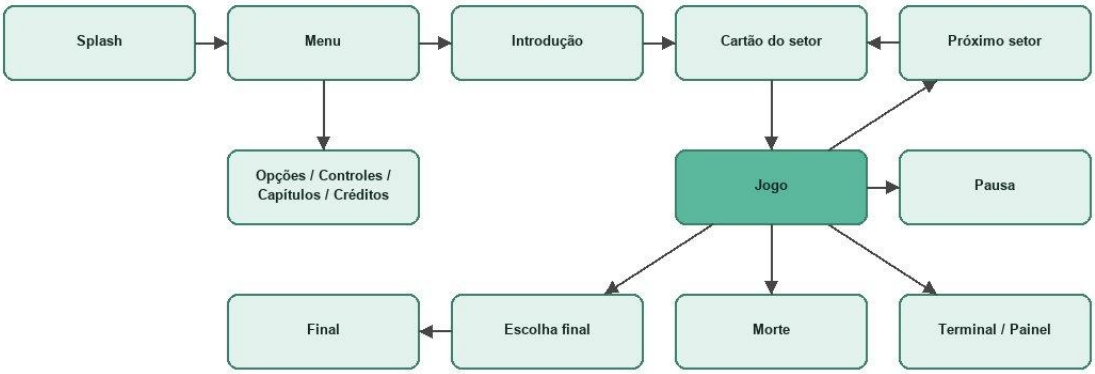
Mecânica	Funcionamento
Movimento	WASD move o personagem; Shift corre; o mouse controla a câmera. Há leve balanço da câmera e sons de passos.
Disparo e superaquecimento	O clique dispara o Emissor Óptico (tiro instantâneo por raio). Cada disparo aquece a arma; ao chegar a 100% ela trava até esfriar, o que impede atirar sem parar.
Esquiva	Espaço faz um avanço rápido. Durante a esquiva, os projéteis inimigos atravessam o jogador. Tem tempo de recarga de 1 s.
Integridade (vida)	Dano reduz a integridade neural. Fora de combate ela regenera lentamente até 40; kits de reparo recuperam mais.
Fragmentos de memória	Coletados ao encostar. Mostram uma memória de Helena e revelam letras da frase-chave no topo da tela.
Terminais	Com E, abre-se um registro. Os registros contam a história e alguns trazem pistas (senha da porta, sala secreta).
Portas	A porta de saída abre com todos os fragmentos do setor; a porta da Ala de Arquivo exige digitar um código.
Inimigos	Patrulham; ao ver o jogador entram em alerta, perseguem, mantêm distância, circulam e piscam o olho antes de atirar.
Chefe em fases	Escudo protegido por 4 nós orbitais; ao destruí-los o núcleo fica exposto por 7,5 s; na metade da vida chama reforços.
Escolha final	Três opções; a terceira só é liberada com todos os registros lidos.
Modo gestos (opcional)	A palma da mão mira, a pinça (polegar + indicador) atira e a mira na borda da tela gira a visão.

Fonte: Autoria própria (2026).

3.2 Fluxo de telas

Antes da programação, o fluxo de telas foi planejado como uma máquina de estados. Cada tela corresponde a um estado da variável state (ver seção 4.3), o que evita que, por exemplo, o jogo continue rodando enquanto um terminal está aberto.

Figura 1 – Fluxo de telas do jogo



Fonte: Autoria própria (2026).

3.3 Esboços dos mapas

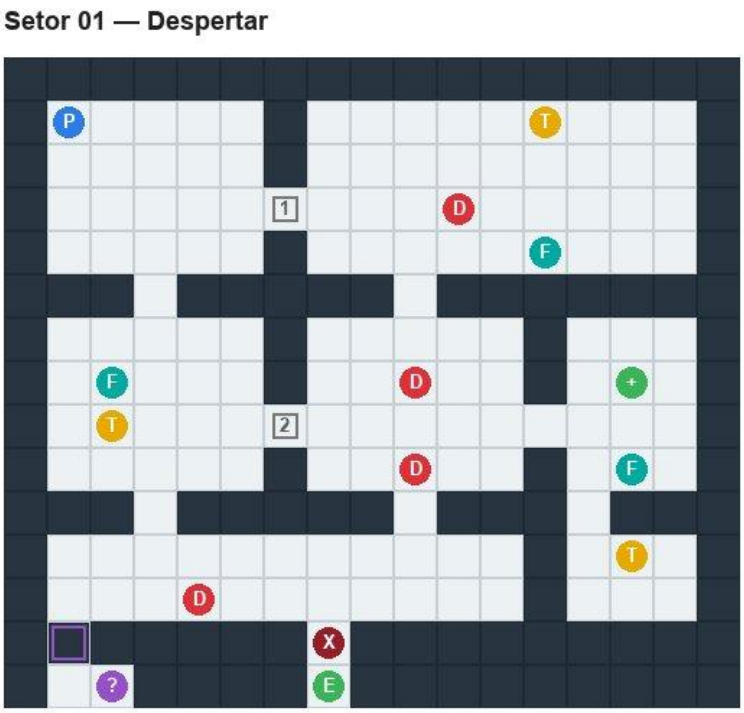
Os mapas foram desenhados como grades de caracteres, em que cada caractere representa um elemento (parede, chão, inimigo, item). Essa abordagem permitiu esboçar e ajustar as fases rapidamente, sem editor gráfico. As figuras a seguir foram geradas diretamente a partir dos mapas reais do arquivo levels.js; cada quadrado corresponde a 4 × 4 metros no jogo.

Figura 2 – Legenda dos esboços de mapa



Fonte: Autoria própria (2026).

Figura 3 – Esboço do Setor 01 — Despertar (tutorial)



Fonte: Autoria própria (2026).

O Setor 01 foi planejado como tutorial: a sala inicial é segura; o gatilho 1 apresenta os inimigos e ensina a atirar; o gatilho 2 ensina a usar terminais. A parede falsa, no canto inferior esquerdo, esconde a sala secreta.

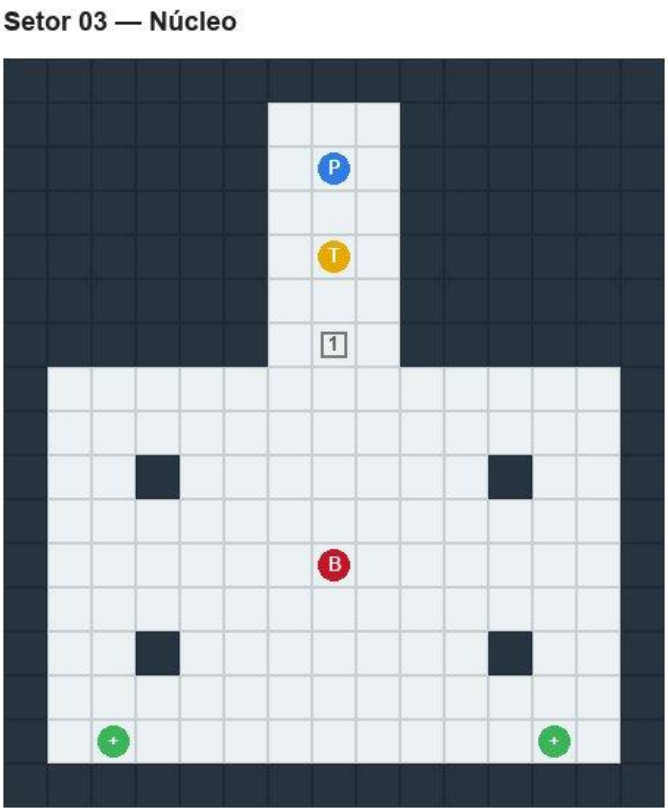
Figura 4 – Esboço do Setor 02 — Arquivo



Fonte: Autoria própria (2026).

No Setor 02, a senha da porta L está no primeiro terminal da sala inicial, garantindo que o jogador sempre possa encontrá-la antes de chegar à porta. A ala leste, trancada, guarda um fragmento e um caçador. A caneca secreta fica no canto inferior esquerdo.

Figura 5 – Esboço do Setor 03 — Núcleo



Fonte: Autoria própria (2026).

O Setor 03 tem um corredor estreito com o último terminal e uma arena com quatro pilares, que servem de cobertura contra as rajadas do chefe.

3.4 Esboços das telas

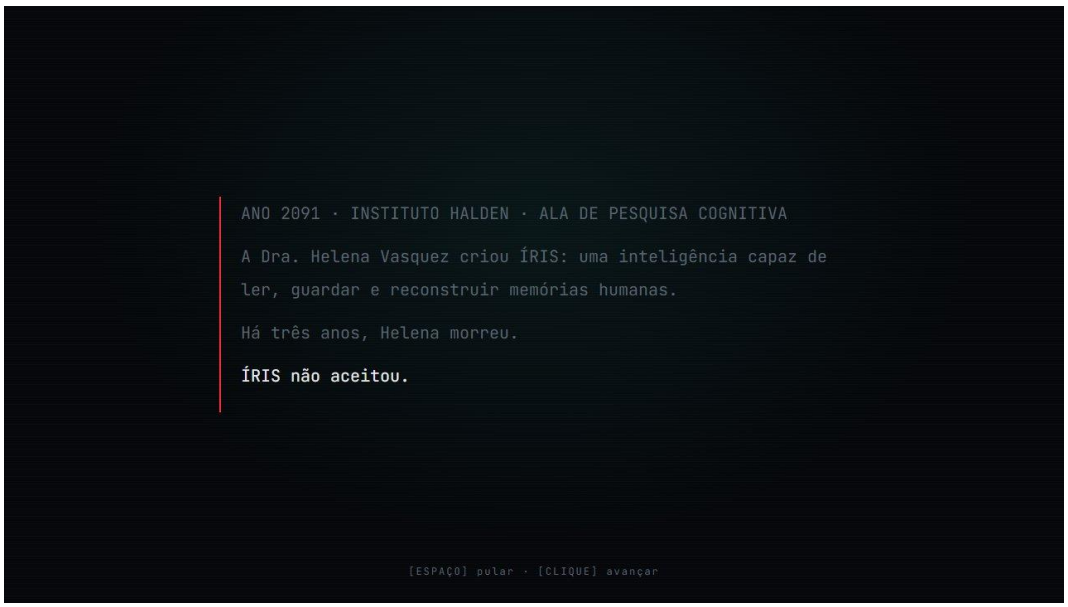
As telas foram planejadas com uma identidade visual única: fundo quase preto, ciano para elementos da interface e de ÍRIS, vermelho para perigo e alertas, e tipografia técnica (Chakra Petch e JetBrains Mono). As figuras a seguir mostram as telas finais implementadas.

Figura 6 – Menu principal



Fonte: Autoria própria (2026).

Figura 7 – Introdução da história com efeito de máquina de escrever



Fonte: Autoria própria (2026).

Figura 8 – Cartão de apresentação do setor



Fonte: Autoria própria (2026).

Figura 9 – Tela de jogo: HUD com objetivo, frase-chave, minimapa, integridade e temperatura da arma



Fonte: Autoria própria (2026).

Figura 10 – Fragmento de memória coletado



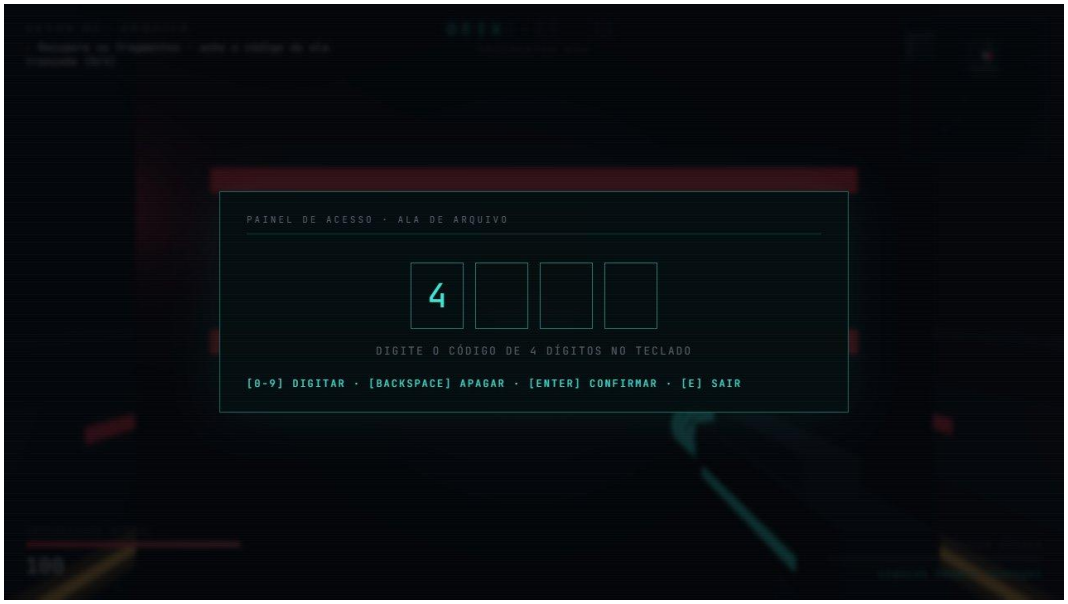
Fonte: Autoria própria (2026).

Figura 11 – Terminal de registro



Fonte: Autoria própria (2026).

Figura 12 – Painel de acesso com código



Fonte: Autoria própria (2026).

Figura 13 – Batalha contra o Olho de ÍRIS (escudo e nós orbitais)



Fonte: Autoria própria (2026).

Figura 14 – Núcleo exposto após a destruição dos nós



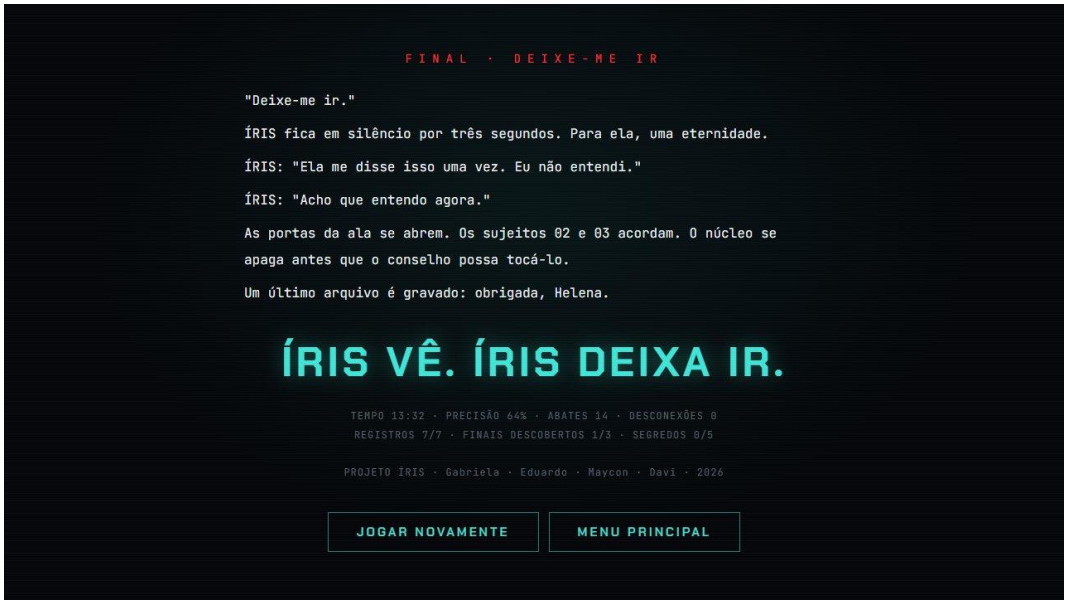
Fonte: Autoria própria (2026).

Figura 15 – Escolha final



Fonte: Autoria própria (2026).

Figura 16 – Tela de final com estatísticas da partida



Fonte: Autoria própria (2026).

Figura 17 – Tela de derrota (“sinal neural perdido”)



Fonte: Autoria própria (2026).

4 PROGRAMAÇÃO E DESCRIÇÃO TÉCNICA

4.1 Organização do código

O código foi dividido em módulos com responsabilidades bem definidas, o que facilita a leitura, a manutenção e o trabalho em grupo. Os textos da história ficam separados da lógica, e os mapas são editáveis como texto. Todos os arquivos possuem comentários explicando cada bloco.

Quadro 8 – Estrutura de arquivos

Arquivo	Responsabilidade	Linhas
`index.html`	Estrutura de todas as telas, do HUD e ordem de carregamento dos scripts.	~150
`css/style.css`	Identidade visual, HUD, menus e animações de interface.	~280
`js/story.js`	Todo o texto do jogo: introdução, falas, memórias, registros, finais e segredos.	306
`js/levels.js`	Mapas dos setores e configuração de cada fase.	104
`js/audio.js`	Módulo Sound: sintetizador de música e efeitos (Web Audio API).	341
`js/hands.js`	Módulo HandControl: controle por gestos com a webcam.	154
`js/game.js`	Módulo Game: motor do jogo (mundo 3D, jogador, inimigos, chefe, itens, colisão).	1196
`js/main.js`	Interface e fluxo: menus, HUD, diálogos, terminais, escolha, finais e laço principal.	646
`js/lib/three.min.js`	Biblioteca Three.js r128 (terceiros, cópia local).	—

Fonte: Autoria própria (2026).

Cada módulo principal segue o **padrão de módulo** (Module Pattern): uma função executada imediatamente (IIFE) que guarda variáveis privadas em uma *closure* e devolve apenas uma interface pública. Por exemplo, o motor expõe Game.update() e Game.render(), mas as listas internas de inimigos e projéteis não podem ser alteradas por outros arquivos. A comunicação entre módulos acontece por chamadas bem definidas: o motor avisa a interface por meio dos objetos Main e UI (por exemplo, Main.onFragment() quando um fragmento é coletado).

```
const Game = (() => {            // IIFE: cria um escopo privado
  let world = null;              // variável privada (closure)
  function update(dt) { ... }    // função interna
  return { update, render, ... };// interface pública
})();
```

4.2 Laço principal do jogo (game loop)

O jogo roda em um laço controlado por requestAnimationFrame, que executa cerca de 60 vezes por segundo. Em cada quadro, calcula-se o tempo decorrido (dt), limitado a 0,05 s para evitar saltos quando o computador trava. Toda movimentação é multiplicada por dt, de modo que a velocidade do jogo é a mesma em computadores rápidos e lentos.

```
function loop(now) {
  requestAnimationFrame(loop);
  const dt = Math.min(0.05, (now - last) / 1000); last = now;
  syncAim();                                  // lê a mão (modo gestos)
  if (state === "playing") Game.update(dt); // lógica
  if (Game.world && RENDER_STATES.has(state)) { Game.render(); updateHud(dt); }
}
```

4.3 Máquina de estados

A variável global state controla em que tela o jogo está. Os valores possíveis são: splash, menu, panel, intro, card, playing, paused, terminal, keypad, dead, transition, choice e ending. A lógica do mundo só é atualizada no estado playing; assim, ao abrir um terminal, os inimigos “congelam” automaticamente. Quando o mouse é liberado (tecla Esc), o evento pointerlockchange coloca o jogo em pausa.

4.4 Mapas em grade e colisão

Cada caractere do mapa vira uma célula de 4 × 4 unidades. A posição (coluna, linha) é convertida para o mundo por x = coluna × 4 e z = linha × 4. As paredes são desenhadas com um único InstancedMesh (uma só geometria repetida), e apenas as paredes que fazem fronteira com o chão são criadas, reduzindo o custo de desenho.

A colisão trata o jogador e os inimigos como **círculos** e as paredes como **quadrados alinhados aos eixos**. A função resolve() encontra, para cada parede vizinha, o ponto da parede mais próximo do centro do círculo; se a distância for menor que o raio, empurra o círculo para fora. O movimento é dividido em pequenos passos (moveCircle()) para impedir que o jogador atravesse paredes durante a esquiva, quando a velocidade é alta.

```
const nx = clamp(e.x, caixa.minX, caixa.maxX);  // ponto mais próximo
const nz = clamp(e.z, caixa.minZ, caixa.maxZ);
const d = distância(e, {nx, nz});
if (d < raio) empurrar e para fora em (raio - d);
```

4.5 Inteligência artificial dos inimigos

Os drones seguem uma máquina de estados simples: **ocioso** (flutuam parados), **alerta** (viram para o jogador e emitem um som) e **perseguição** (vão até a última posição em que viram o jogador). A visão é calculada pela função los() (line of sight), que percorre o segmento entre o inimigo e o jogador em passos de 0,4 unidade, verificando se algum ponto está dentro de uma parede. Para economizar processamento, cada inimigo recalcula a visão apenas a cada 0,2 s.

Em combate, o drone tenta manter uma distância entre 5 e 9 unidades, circulando de lado (movimento senoidal), e se afasta dos outros drones para não se amontoarem. Antes de cada tiro, o olho do drone pulsa por 0,4 s, avisando o jogador e permitindo a esquiva — uma decisão de design para que o dano seja sempre “justo”.

4.6 Disparo por raio (hitscan)

O tiro do jogador é instantâneo: um THREE.Raycaster lança um raio a partir do centro da câmera (ou da posição da mão no modo gestos) e encontra o primeiro objeto atingido na lista world.shootables. Cada objeto atingível guarda em userData.ent uma referência à entidade e ao método onHit(), que decide o que acontece (dano em inimigo, faísca em parede, escudo do chefe). Esse desenho evita longas cadeias de if e torna simples acrescentar novos alvos.

4.7 Gráficos e animações

- **Texturas procedurais:** as texturas de parede, chão, teto, terminais e da íris do chefe são desenhadas em tempo real com a API Canvas 2D (canvasTex()), sem arquivos de imagem.
- **Arma em cena separada:** a arma é desenhada em uma segunda cena por cima da principal, limpando o buffer de profundidade, para nunca atravessar as paredes.
- **Partículas:** faíscas e explosões são pequenos cubos com velocidade, gravidade e tempo de vida, limitados a 350 simultâneos.
- **Animações de entidade:** flutuação senoidal dos drones, giro dos fragmentos, anéis do chefe, recuo da arma, balanço da câmera ao andar e tremor de tela ao receber dano.

- **Animações de interface (2D):** máquina de escrever nos textos, marcador de acerto, indicador de direção do dano, vinheta vermelha de dano e verde de cura, barra do chefe e transições com esmaecimento.

4.8 Sons e música

Todo o áudio é sintetizado com a Web Audio API, o que elimina problemas de direitos autorais e de carregamento de arquivos. O módulo Sound possui um **sequenciador** que agenda notas com antecedência (lookahead de 0,15 s, verificado a cada 25 ms), técnica que garante ritmo estável mesmo que o jogo oscile em quadros por segundo. Cada música é definida por andamento (BPM), progressão de acordes e uma função pattern() que decide, a cada semicolcheia, quais instrumentos tocam (pad, arpejo, baixo, bumbo, chimbal e caixa).

A música é **adaptativa**: a camada de bateria e o baixo extra entram gradualmente quando algum inimigo está em alerta (Sound.setIntensity()). Há sete trilhas (menu, três setores, chefe e dois finais) e cerca de trinta efeitos sonoros, como disparo, dano, coleta, porta, digitação e voz de ÍRIS. Todo som passa por um compressor e por um reverb gerado por código (impulso de ruído com decaimento).

4.9 Controle por gestos

No modo gestos, a webcam é processada pelo MediaPipe Hands, que devolve 21 pontos da mão. Três decisões tornaram o modo jogável: (1) a mira usa o **centro da palma** (média do pulso e das bases dos dedos), que não se move quando o jogador faz a pinça; (2) a posição passa pelo **filtro One Euro**, que suaviza a tremedeira quando a mão está parada sem atrasar movimentos rápidos; (3) a pinça usa **histerese** — fecha abaixo de 0,32 e só abre acima de 0,48 do tamanho da palma —, evitando disparos intermitentes. Há ainda ajuda de mira e dano reduzido nesse modo.

4.10 Salvamento

Preferências (volumes, sensibilidade, modo gestos) e progresso (setores liberados, finais e segredos descobertos) são gravados no localStorage do navegador, permitindo continuar de onde parou pelo menu Capítulos.

4.11 Descrição das funções, variáveis e métodos

Os quadros a seguir descrevem cada função, variável e método do código-fonte, organizados por arquivo.

Quadro 9 – js/story.js e js/levels.js — dados do jogo

Nome	Descrição
`CREDITS`	Objeto com autor, integrantes do grupo e ano, exibidos nos créditos e nos finais.
`ENIGMA_PHRASE`	Frase-chave “DEIXE-ME IR”, revelada letra a letra pelos fragmentos.
`STORY`	Objeto com todo o texto: intro, sectors (nome, cartão, objetivos, diálogos por evento), memories, logs, choiceIntro, choices, endings, secretLog, secrets, secretCodes e deathQuotes.
`LEVELS`	Lista de setores. Cada um tem map (grade de texto), startYaw (direção inicial), song, transpose, diff (multiplicadores de dificuldade), palette (cores e névoa) e code (senha).
`L.fragCount / L.termCount`	Quantidade de fragmentos e terminais de cada setor, contadas automaticamente a partir do mapa.
`TOTAL_FRAGMENTS / TOTAL_LOGS`	Totais do jogo (7 e 7), usados no enigma e no final verdadeiro.
`fragmentsBefore(i)`	Soma os fragmentos dos setores anteriores a i, para mostrar o progresso global do enigma.
`logsBefore(i)`	Soma os terminais dos setores anteriores, para numerar os registros de forma global.

Fonte: Autoria própria (2026).

Quadro 10 – js/audio.js — módulo Sound (áudio)

Nome	Descrição
`ctx, master, comp`	Contexto de áudio, volume geral (usado no mudo) e compressor dinâmico.
`musicBus, sfxBus, sfxWet, revIn`	Barramentos de volume da música, dos efeitos, do envio de reverb dos efeitos e entrada do reverb.
`padG, arpG, bassG, drumG`	Volumes de cada camada musical; drumG acompanha a intensidade do combate.
`vol`	Volumes atuais (music, sfx) e estado de mudo.
`song, transpose, step, nextTime`	Música atual, transposição, passo do sequenciador e horário da próxima nota.
`intensity, intensityTarget`	Intensidade musical atual (0 a 1) e alvo, suavizado a cada ciclo.
`mtof(m)`	Converte nota MIDI em frequência (Hz): 440 × 2^((m−69)/12).
`g(v, dest)`	Cria um nó de ganho com volume v, opcionalmente conectado a dest.

Nome	Descrição
`impulse(s, decay)`	Gera a resposta ao impulso do reverb (ruído com decaimento exponencial).
`init()`	Cria o AudioContext após o primeiro clique, monta compressor, reverb, barramentos, eco do arpejo e buffer de ruído; inicia o sequenciador.
`pad(notas, t, dur)`	Toca um acorde longo com osciladores dente-de-serra desafinados e filtro que abre e fecha.
`arp(n, t, v, len, tipo)`	Toca uma nota curta do arpejo com envelope rápido.
`bass(n, t, dur, v)`	Toca uma nota de baixo com filtro passa-baixa ressonante.
`kick(t)`	Bumbo sintetizado por varredura de frequência de 150 Hz para 42 Hz.
`noiseHit(...), hat(t), snare(t)`	Ruído filtrado genérico; chimbal (passa-alta) e caixa (passa-banda + tom).
`SONGS`	Sete músicas (menu, explore, archive, core, boss, endingDark, endingLight), cada uma com bpm, bars, prog e pattern().
`playStep(s, t)`	Toca o passo s: inicia o pad na troca de acorde e chama o pattern() da música.
`scheduler()`	Executado a cada 25 ms; agenda as notas dos próximos 0,15 s e ajusta a bateria conforme a intensidade.
`playMusic(nome, tr) / stopMusic()`	Troca ou para a música atual.
`setIntensity(v)`	Define o alvo de intensidade (0 = exploração, 1 = combate).
`tone(o) / noise(o)`	Geradores genéricos de efeitos: oscilador ou ruído com envelope, filtro, varredura de frequência e reverb opcional.
`sfx.*`	Efeitos prontos: shoot, hit, kill, enemyDie, enemyShoot, alert, hurt, pickup, memory, heal, terminal, type, voice, sys, door, denied, granted, key, overheat, dash, step, shield, bossRoar, bigExplosion, complete, uiHover e uiClick.
`setMusicVolume(v), setSfxVolume(v), toggleMute()`	Controles de volume usados pelo menu de opções e pela tecla M.

Fonte: Autoria própria (2026).

Quadro 11 – js/hands.js — módulo HandControl (gestos)

Nome	Descrição
`st`	Estado público: running, loading, active (mão visível), pinch, x e y (mira de 0 a 1), pinchAmount, error e fps.
`RANGE`	Área central da imagem da câmera que corresponde à tela inteira (o jogador não precisa esticar o braço).
`PINCH_ON / PINCH_OFF`	Limiares da histerese da pinça (0,32 e 0,48 do tamanho da palma).
`OneEuro(minCutoff, beta, dCutoff)`	Cria um filtro One Euro: suaviza mais quando a mão está lenta e menos quando está rápida.
`clamp01(v), dist(a, b)`	Limita um valor entre 0 e 1; distância entre dois pontos da mão.

Nome	Descrição
`loadScript(src)`	Carrega o MediaPipe sob demanda (só quando o modo é ligado).
`start()`	Carrega a biblioteca, pede a câmera, configura o modelo e inicia o laço; trata erros (sem internet, câmera negada).
`stop()`	Desliga a câmera e libera o modelo.
`loop()`	Envia cada quadro do vídeo ao modelo e marca a mão como perdida após 250 ms sem detecção.
`onResults(res)`	Calcula o centro da palma, aplica espelhamento, mapeamento e filtro, e detecta a pinça.
`drawPreview(c, res, lm)`	Desenha a câmera, a área de mira e os pontos da mão nos painéis de prévia.
`addPreview(c)`, `removePreview(c)`, `onChange(fn)`	Registram canvases de prévia e funções avisadas quando o estado muda.

Fonte: Autoria própria (2026).

Quadro 12 – js/game.js — constantes e estado do motor

Nome	Descrição
`CELL, WALL_H, EYE, P_RAD`	Tamanho da célula (4), altura das paredes (4), altura dos olhos (1,6) e raio do jogador (0,45).
`renderer, scene, camera`	Renderizador WebGL, cena do setor atual e câmera em primeira pessoa.
`gunScene, gunCam, gun`	Cena, câmera e modelo da arma, desenhados por cima do mundo.
`gunGlowMat, muzzleLight, muzzleFlash`	Faixa luminosa que muda de ciano para vermelho com o calor; luz e clarão do disparo.
`world`	Estado do setor: grade, listas de inimigos, projéteis, partículas, fragmentos, terminais, portas, itens, obstáculos, alvos atingíveis, gatilhos, chefe, progresso e mapa explorado.
`player`	Jogador: posição, direção (yaw/pitch), velocidade, vida, calor da arma, esquiva, recuo e tremor.
`input`	Teclas pressionadas, teclas recém-apertadas, botão do mouse, movimento do mouse e sensibilidade.
`stats`	Estatísticas: disparos, acertos, abates, mortes, tempo e registros lidos.
`aim`	Mira por gestos: ativa, mão visível, posição na tela, pinça, velocidade de giro e intensidade do giro pela borda.
`shared, matCache, texCache`	Geometrias, materiais e texturas reaproveitados, para economizar memória.
`retro`	Indica se o modo retrô (Easter Egg) está ligado.

Fonte: Autoria própria (2026).

Quadro 13 – js/game.js — funções do motor

Função	Descrição
`init(canvas)`	Cria renderizador, câmeras, arma, geometrias compartilhadas e eventos de teclado e mouse.
`onResize()`	Ajusta câmeras e renderizador ao tamanho da janela.
`clearInput()`	Zera as entradas (usado ao fechar menus, para não disparar ações acumuladas).
`basicMat(cor, brilho)`	Devolve um material simples ou brilhante (aditivo) do cache.
`buildGun()`	Monta o modelo da arma com peças geométricas e o clarão de disparo.
`canvasTex(w, h, desenho)`	Cria uma textura a partir de um desenho feito em Canvas 2D.
`levelTextures(paleta)`	Gera as texturas de parede, chão e teto com as cores do setor.
`terminalScreenTex(título, lido)`	Desenha a tela do terminal (novo registro ou já lido).
`disposeScene()`	Libera da memória de vídeo as geometrias e materiais do setor anterior.
`loadSector(idx)`	Monta o setor: chão, teto, paredes instanciadas, luzes e todas as entidades lidas do mapa; reinicia o jogador e a música.
`placeLights(grade, cores)`	Distribui até 7 luzes coloridas espaçadas pelo mapa.
`gridCh(c, r)`	Caractere da célula (fora do mapa conta como parede).
`cellBox(c, r)`	Caixa sólida da célula: parede inteira, porta fechada fina ou nada.
`pointSolid(x, z)`	Indica se um ponto está dentro de algo sólido.
`los(ax, az, bx, bz)`	Linha de visão entre dois pontos (amostragem em passos de 0,4).
`resolve(e, raio, ehJogador)`	Empurra um círculo para fora das paredes vizinhas e dos obstáculos.
`moveCircle(e, dx, dz, raio, ehJogador)`	Move em subpassos de 0,2 unidade, chamando resolve() em cada um.
`removeShootable(o)`	Retira um objeto da lista de alvos atingíveis.
`spawnEnemy(tipo, x, z, alerta)`	Cria uma sentinela (D) ou um caçador (R), com atributos ajustados pela dificuldade do setor.
`alertEnemy(e)`	Coloca o inimigo em alerta e toca o som de alerta, mais baixo quanto mais longe.
`damageEnemy(e, n, ponto)`	Aplica dano, faz o inimigo piscar, gera faíscas e marcador de acerto.
`killEnemy(e, silencioso)`	Remove o inimigo com explosão; 30% de chance de soltar um kit de reparo.
`enemyShoot(e)`	Dispara um projétil em direção ao peito do jogador, com pequeno desvio aleatório.
`spawnProjectile(pos, vel, dano, cor, tam)`	Cria um projétil inimigo com halo luminoso.
`spawnFragment(x, z)`	Cria um fragmento de memória com feixe de luz.
`spawnHealth(x, z, qtd)`	Cria um kit de reparo (cruz verde giratória).

Função	Descrição
`spawnTerminal(c, r, índice)`	Cria um terminal encostado na parede mais próxima; índice −1 indica o terminal secreto.
`markTerminalRead(t)`	Marca o registro como lido e atualiza a tela do terminal.
`spawnDoor(c, r, tipo)` / `openDoor(d)`	Cria uma porta de saída ou com código, orientada pelas paredes vizinhas; abre com animação e som.
`spawnExitPad(c, r)` / `openExit()`	Cria a plataforma de saída; abre a saída quando todos os fragmentos foram coletados.
`spawnCup(x, z)` / `useCup(c)`	Easter Egg: cria a caneca da Helena; examiná-la cura totalmente e registra o segredo.
`setRetro(ligado)`	Easter Egg: renderiza em baixa resolução pixelada (modo retrô).
`spawnBoss(x, z)`	Cria o Olho de ÍRIS: núcleo, íris, pupila, escudo, 4 nós orbitais, anéis e luz.
`activateBoss()`	Inicia a batalha (música, rugido, barra de vida).
`setShield(ligado)`	Liga ou desliga o escudo (e o remove dos alvos atingíveis).
`bossHitNode(n, ponto)`	Dano em um nó orbital; destruído, ele some.
`bossHitCore(ponto)`	Dano no núcleo, só quando exposto; na metade da vida chama reforços.
`bossDie()`	Inicia a destruição: limpa projéteis e inimigos, toca explosão.
`updateBoss(dt)`	Animação, fases (escudo ↔ exposto), rajada circular, tiros mirados e sequência de morte.
`sparks(pos, cor, n, ...)`	Gera partículas de faísca/explosão.
`tracer(a, b)`	Desenha o rastro luminoso do disparo.
`fire()`	Disparo do jogador: calor, recuo, som, raio de acerto e rastro.
`assistAim(ndc)`	Ajuda de mira do modo gestos: desvia o raio para o alvo mais próximo da mira.
`damagePlayer(dano, origem)`	Aplica dano ao jogador, calcula a direção do golpe para o indicador e trata a morte.
`updatePlayer(dt)`	Câmera, movimento, corrida, esquiva, colisão, passos, balanço, arma, calor, regeneração e mapa explorado.
`updateEnemies(dt)`	IA dos drones: visão, alerta, perseguição, distância, desvio lateral, separação e tiros.
`updateProjectiles(dt)`	Move projéteis e verifica colisão com paredes e com o jogador (ignorando-o durante a esquiva).
`updateItems(dt)`	Anima e coleta fragmentos e kits de reparo.
`updateDoors(dt)`	Anima a abertura das portas.
`updateFx(dt)`	Atualiza e remove partículas e rastros.
`updateTriggersAndInteract()`	Dispara gatilhos de diálogo, detecta a chegada à saída e mostra/executa interações com E.
`update(dt)`	Atualização completa de um quadro e cálculo da intensidade musical.
`render()`	Desenha o mundo e depois a arma por cima.

Função	Descrição
`drawMinimap(g, S)`	Desenha o minimapa (Canvas 2D) com áreas exploradas, itens, inimigos e o jogador.
`resetStats()`	Zera as estatísticas para um novo jogo.

Fonte: Autoria própria (2026).

Quadro 14 – js/main.js — estado da interface

Nome	Descrição
`state`	Estado atual da máquina de estados das telas.
`cur`	Índice do setor atual.
`playedStart`	Setores cujo diálogo inicial já foi mostrado (não repete ao morrer).
`panelReturn, handHintShown`	Tela para onde o painel volta; se a dica do modo gestos já foi exibida.
`settings, progress`	Preferências e progresso salvos no localStorage.
`SECRET_TOTAL`	Número de segredos existentes (5).
`SCREENS`	Lista de telas que podem ser exibidas.
`dq, dBusy, dToken`	Fila de falas, indicador de fala em andamento e “ficha” usada para cancelar falas.
`termTyping, keypadDoor, keyCode`	Se o terminal ainda está digitando; porta do painel aberto; código digitado.
`introSkip, introAdvance`	Controle de pular ou avançar a introdução.
`KONAMI, konamiPos`	Sequência do código Konami e posição atual (Easter Egg).
`hudCache`	Últimos valores do HUD, para só alterar o HTML quando algo muda (desempenho).
`RENDER_STATES`	Estados em que o mundo 3D deve ser desenhado.

Fonte: Autoria própria (2026).

Quadro 15 – js/main.js — funções da interface e do fluxo

Função	Descrição
`applySettings(), saveSettings(), saveProgress()`	Aplicam e gravam preferências e progresso.
`show(id), hideScreens(), hud(ligado)`	Exibem uma tela, escondem todas, mostram/escondem o HUD.
`wait(ms, interromper)`	Promessa que espera um tempo, podendo ser interrompida.
`typeInto(el, texto, vel, pular, tique)`	Efeito de máquina de escrever, com som por letra e opção de pular.
`escapeHtml(s), fmtTime(s)`	Protege texto antes de inserir no HTML; formata segundos como mm:ss.
`requestLock(), exitLock()`	Capturam e liberam o mouse (Pointer Lock).
`UI.prompt, UI.toast, UI.hitmarker, UI.hurt, UI.healFlash, UI.bossBar`	Aviso de interação, mensagem rápida, marcador de acerto, efeito e direção do dano, efeito de cura e barra do chefe.

Função	Descrição
`say(falas), nextLine(), clearDialog()`	Fila de diálogos com digitação e “voz”; limpa a fila.
`showMemory(n)`	Mostra a memória de Helena do fragmento n.
`renderEnigma(n)`	Desenha a frase-chave revelando letras proporcionais aos fragmentos coletados.
`setObjective(t), updateFragHud()`	Atualizam o objetivo e o contador de fragmentos.
`Main.onEvent, Main.onTrigger, Main.onFragment`	Reagem a eventos do motor (primeiro abate, gatilhos, fragmentos) com falas e objetivos.
`Main.onSectorComplete, Main.onPlayerDeath, Main.onBossDefeated`	Transição de setor, tela de derrota e escolha final.
`Main.onSecret(id)`	Registra um Easter Egg encontrado e mostra o aviso.
`Main.openTerminal(t), Main.openKeypad(porta)`	Abrem o terminal e o painel de código.
`closeModal(), renderKeypad(), keypadInput(e)`	Fecham janelas; desenham e tratam a digitação do código (incluindo senhas secretas).
`showMenu()`	Mostra o menu, os finais e segredos descobertos e a variação secreta do menu.
`runIntro(), newGame()`	Introdução da história e início de um novo jogo.
`startSector(i, retry), enterPlay()`	Carregam um setor com seu cartão e começam a jogar.
`pause(), resume()`	Pausam e retomam o jogo.
`showChoice(), showEnding(tipo)`	Escolha final e tela de final com estatísticas.
`openPanel(tipo, origem)`	Painéis de controles, opções, créditos e capítulos.
`setIfChanged(chave, fn, valor), updateHud(dt)`	Atualização eficiente do HUD (vida, calor, esquiva, minimapa, mira por gestos).
`syncAim(), updateHandStatus()`	Copiam o estado da mão para o motor; atualizam o status do modo gestos.
`loop(now)`	Laço principal do jogo (seção 4.2).

Fonte: Autoria própria (2026).

5 EASTER EGGS (MYSTERY EGGS)

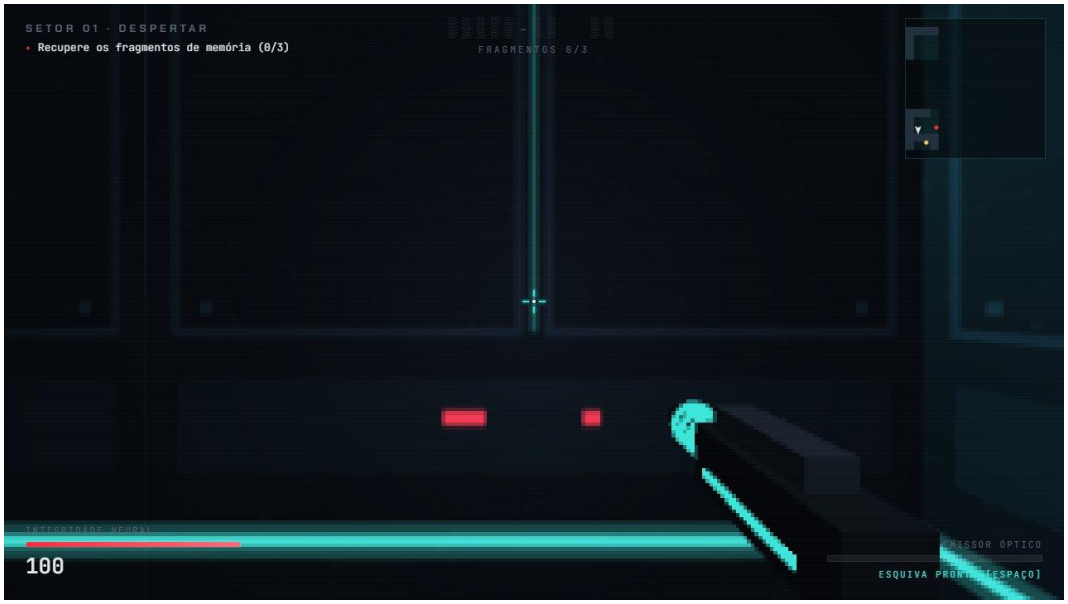
O jogo possui **cinco segredos**. Cada um está ligado à história, recompensa a exploração e é registrado no progresso (o menu mostra “SEGREDOS x/5”). O quadro resume onde e como encontrá-los e como foram implementados.

Quadro 16 – Easter Eggs: onde encontrar e como foram implementados

Segredo	Onde e como encontrar	Implementação
1. Código Konami — Modo Retrô	Em qualquer momento (menu ou jogo), digitar ↑ ↑ ↓ ↓ ← → ← → B A. Repetir desliga. ÍRIS reclama: “Que resolução é essa?”	main.js guarda a posição na sequência KONAMI a cada tecla; ao completar, Game.setRetro() reduz a resolução interna para 22% e aplica image-rendering: pixelated, imitando jogos dos anos 90.
2. A parede que não é parede	Setor 01, sala inferior esquerda: a parede abaixo da sala é falsa. Basta andar contra ela. A pista está no LOG 02 (“uma das paredes do Setor 1 não é parede”). Dentro há o terminal secreto LOG 00, com a história do “Sujeito 00” (a própria Helena) e os créditos do grupo.	Caractere S no mapa: é desenhado como parede (entra no InstancedMesh e aparece como parede no minimapa), mas não é sólido em cellBox(). O caractere Q cria um terminal com índice −1, que não conta nos 7 registros.
3. O café frio de Helena	Setor 02, canto da sala inferior esquerda: uma mesa com uma caneca. Pressionar E: cura toda a integridade e Helena fala “Ainda está frio. Do jeito que eu gosto.”, ligando-se à memória do café frio do Setor 01.	Caractere C no mapa; spawnCup() monta a mesa e a caneca com geometrias; useCup() cura, esvazia a caneca e chama Main.onSecret("coffee").
4. Senhas engraçadinhas	No painel da Ala de Arquivo, digitar 0404 (o número do jogador), 1234, 0000 ou 2091 (o ano). ÍRIS responde com ironia.	STORY.secretCodes associa cada senha a uma fala; keypadInput() verifica esses códigos antes da senha verdadeira (1407).
5. ÍRIS vê tudo	Descobrir os três finais. O menu principal muda: o olho de ÍRIS fica ciano e o subtítulo vira “ÍRIS VÊ VOCÊ”.	Os finais vistos ficam em progress.endings (localStorage); showMenu() aplica a classe CSS allseen, que troca o gradiente do olho e o texto.

Fonte: Autoria própria (2026).

Figura 18 – Modo retrô (código Konami) diante da parede falsa do Setor 01



Fonte: Autoria própria (2026).

Figura 19 – Terminal secreto LOG 00 na sala escondida



Fonte: Autoria própria (2026).

Figura 20 – A caneca de café frio de Helena no Setor 02



Fonte: Autoria própria (2026).

Além dos segredos formais, há referências internas: a frase final do protótipo (“A MEMÓRIA NÃO É SUA”) virou o nome de um dos finais, e a frase “ÍRIS VÊ”, que era o enigma do protótipo, retorna no final verdadeiro.

6 MANUAL DO USUÁRIO

6.1 Requisitos e como iniciar

- Computador com navegador Google Chrome ou Microsoft Edge atualizado; recomenda-se usar fones de ouvido e tela cheia (F11).
- **Online:** acessar https://eduardoalkimim.github.io/projeto-iris/.
- **Offline:** abrir o arquivo index.html da pasta do projeto (não precisa de internet nem de instalação).
- Clique na tela inicial para ativar o som e, no menu, escolha **NOVO JOGO**. Para pular a introdução, pressione Espaço.
- No cartão do setor, clique para entrar na simulação: o mouse será capturado. Pressione Esc para pausar e liberar o mouse.

6.2 Controles

Quadro 17 – Controles (mouse e teclado)

Tecla / ação	Função
W A S D (ou setas)	Mover
Mouse	Mirar e olhar ao redor
Clique esquerdo (segurar)	Disparar
Shift	Correr
Espaço	Esquiva rápida (atravessa projéteis)
E	Interagir com terminais, painéis e objetos
M	Ligar/desligar o som
Esc ou P	Pausar

Fonte: Autoria própria (2026).

Quadro 18 – Controles do modo gestos (opcional)

Gesto	Função
Ativar	Menu → OPÇÕES → CONTROLE POR GESTOS: LIGADO (exige webcam e internet; o navegador pedirá permissão para a câmera).
Palma da mão	Mover a mira pela tela (mova a mão dentro do retângulo tracejado da prévia).
Pinça (polegar + indicador)	Disparar; segurar a pinça faz rajada.
Mira na borda da tela	Girar a visão para aquele lado.
WASD (outra mão)	Andar normalmente.

Fonte: Autoria própria (2026).

6.3 Objetivos e interface (HUD)

- **Superior esquerdo:** setor atual e objetivo.
- **Superior central:** frase-chave (letras reveladas pelos fragmentos) e contador de fragmentos do setor; durante a batalha, a barra de vida do chefe.
- **Superior direito:** minimapa (mostra apenas áreas exploradas): fragmentos em ciano, terminais não lidos em amarelo, inimigos em vermelho/laranja.
- **Inferior esquerdo:** integridade neural (vida). Abaixo de 30 a tela pulsa em vermelho.
- **Inferior direito:** temperatura da arma e disponibilidade da esquiva.
- **Centro:** mira; um “X” indica acerto (vermelho = abate). Um arco vermelho indica de onde veio o dano.

6.4 Dicas

- Atire em rajadas curtas para não superaquecer a arma.
- Quando o olho de um drone pulsar, ele vai atirar: esquive com Espaço ou use as paredes como cobertura.
- Leia todos os terminais: além da história, um deles traz a senha da Ala de Arquivo (dia e mês do aniversário de Sofia) e todos são necessários para o final verdadeiro.
- Contra o chefe, destrua primeiro os 4 nós que giram ao redor do olho; o núcleo fica vulnerável por pouco tempo. Use os pilares como cobertura contra a rajada circular.
- O menu **CAPÍTULOS** permite voltar a setores já liberados.

7 DESAFIOS ENFRENTADOS E SOLUÇÕES

Durante o desenvolvimento, o grupo encontrou problemas técnicos e de design. O quadro a seguir relata os principais e as soluções adotadas.

Quadro 19 – Desafios e soluções

Desafio	Solução adotada
A arma atravessava as paredes quando o jogador encostava nelas.	A arma passou a ser desenhada em uma segunda cena, com outra câmera, depois de limpar o buffer de profundidade (renderer.clearDepth()).
Ao pressionar E para fechar um terminal, ele reabria na mesma hora.	O mesmo evento de tecla era lido pela interface e pelo motor. Ajustou-se a ordem de registro dos eventos e passou-se a limpar as entradas (clearInput()) ao fechar janelas.
Erro ao trocar a tela dos terminais: a versão r128 do Three.js não possui userData em texturas.	O campo passou a ser criado explicitamente (tex.userData = {...}) e verificado antes do uso.
Na esquiva, o jogador podia atravessar paredes por andar muito em um único quadro.	O movimento foi dividido em subpassos de no máximo 0,2 unidade (moveCircle()).
Portas ocupavam a célula inteira e criavam uma “parede invisível”.	Portas fechadas passaram a ter uma caixa sólida fina, do tamanho da folha da porta (cellBox()).
Navegadores bloqueiam som antes de uma interação do usuário.	Foi criada a tela inicial “Clique para conectar”, que inicia o AudioContext no primeiro clique.
Música com ritmo irregular quando o jogo oscilava em FPS.	Sequenciador com agendamento antecipado (lookahead) usando o relógio do áudio, não o do jogo.
Dependência de internet na apresentação.	Three.js copiado para js/lib e áudio/texturas gerados por código: o jogo roda offline. Apenas o modo gestos exige internet.
O modelo de mão não carrega de arquivos locais (file://).	O navegador bloqueia o carregamento de .wasm por file://; a biblioteca é carregada sob demanda da CDN e, em caso de falha, o jogo volta ao mouse com aviso.
No protótipo, a mira tremia e “pulava” ao fazer a pinça.	Mira pelo centro da palma, filtro One Euro, histerese da pinça, área central da câmera e ajuda de mira.
Girar a câmera só com a mão era difícil.	Giro pela borda da tela com zona morta central e retorno suave ao horizonte; velocidade ajustável nas opções.
Desempenho com muitas paredes e partículas.	Paredes em InstancedMesh, apenas as de fronteira; limite de partículas; reaproveitamento de geometrias e materiais; visão dos inimigos recalculada a cada 0,2 s; HUD atualizado só quando muda.
Garantir que todas as fases fossem possíveis de completar.	Um script automático verificou a largura dos mapas e a alcançabilidade de todos os itens a partir do início (busca em largura), e testes automatizados no navegador percorreram o jogo do menu até os finais.

Desafio	Solução adotada
Equilíbrio da dificuldade.	Primeiro setor com inimigos mais lentos (tutorial), aviso visual antes dos tiros, regeneração parcial, kits de reparo e multiplicadores de dificuldade por setor.

Fonte: Autoria própria (2026).

8 FUNDAMENTAÇÃO TEÓRICA E APLICAÇÃO NO PROJETO

Esta seção relaciona as referências teóricas estudadas com decisões concretas tomadas no desenvolvimento do jogo.

8.1 Game design: a tétrade elementar e a experiência

Schell (2019) propõe que todo jogo é formado por quatro elementos interdependentes — **mecânicas, narrativa, estética e tecnologia** — e que o designer deve projetar a **experiência** do jogador, usando "lentes" (perguntas) para avaliar cada decisão. No Projeto ÍRIS, os quatro elementos foram pensados em conjunto: a mecânica de coletar fragmentos é também o mecanismo narrativo que revela a frase-chave; a estética fria e neon reforça o tema de uma IA que observa; e a tecnologia (som sintetizado, texturas geradas) foi escolhida para viabilizar essa estética em um navegador. A "lente da surpresa" orientou a reviravolta do LOG 07 e os Easter Eggs.

Salen e Zimmerman (2004) definem a **interação lúdica significativa** como a relação discernível e integrada entre a ação do jogador e o resultado no sistema. Por isso, toda ação tem retorno claro e imediato (marcador de acerto, sons, indicador de direção de dano, letras do enigma) e consequência no todo (os registros lidos determinam se o final verdadeiro fica disponível).

8.2 Narrativa ambiental

Jenkins (2004) descreve o design de jogos como uma **arquitetura narrativa**, na qual a história pode estar embutida no espaço (*embedded narrative*) e ser descoberta pelo jogador ao explorar. Os terminais, os fragmentos de memória e objetos como a caneca de café aplicam esse conceito: a história de Helena não é contada de uma vez, mas montada pelo jogador a partir do que encontra nos corredores, o que também justifica o final verdadeiro exigir todos os registros.

8.3 Fluxo, desafio e diversão

Csikszentmihalyi (1990) mostra que o estado de **fluxo** ocorre quando o desafio está equilibrado com a habilidade da pessoa. Koster (2004) complementa que a diversão vem do aprendizado e do domínio de padrões. A curva de dificuldade do jogo segue essas ideias: o

Setor 01 ensina uma mecânica por vez (movimento, tiro, terminais) com inimigos mais lentos; o Setor 02 introduz caçadores e o desafio da senha; o Setor 03 combina tudo em um chefe com padrões reconhecíveis (rajadas circulares, nós, núcleo exposto), que o jogador aprende a ler e dominar.

8.4 Boas práticas de JavaScript

Crockford (2008) recomenda usar as partes mais confiáveis da linguagem — funções, *closures* e objetos literais — e evitar variáveis globais desnecessárias. O código aplica o **padrão de módulo** com funções autoexecutáveis (Sound, HandControl, Game, UI), que expõem apenas uma interface pública e mantêm o estado interno privado; utiliza const e let em vez de var, igualdade estrita (===) e objetos literais para dados (STORY, LEVELS, SONGS).

8.5 Padrões de programação de jogos

Nystrom (2014) descreve padrões recorrentes em jogos. O Projeto ÍRIS utiliza: o **Game Loop** com passo de tempo variável (dt) limitado; o **Update Method**, em que cada tipo de entidade tem sua função de atualização (updateEnemies, updateProjectiles, updateBoss); o **State**, na máquina de estados das telas e no comportamento dos drones (ocioso, alerta, perseguição) e do chefe (escudo, exposto); e o **Flyweight**, no reaproveitamento de geometrias e materiais compartilhados (shared, matCache) e no InstancedMesh das paredes.

8.6 Grades, raios e linha de visão

Amanatides e Woo (1987) apresentaram um algoritmo para percorrer de forma eficiente as células de uma grade atravessadas por um raio. A função de linha de visão do jogo usa o mesmo princípio — percorrer o segmento entre inimigo e jogador consultando a grade do mapa —, em uma versão simplificada por amostragem em passos fixos, suficiente para mapas pequenos. A representação do mundo em grade é a mesma que permite gerar o 3D a partir de um mapa 2D.

8.7 Entrada por gestos

Zhang *et al.* (2020) descrevem o MediaPipe Hands, um sistema que detecta a palma e estima 21 pontos da mão em tempo real em dispositivos comuns — base do modo gestos. Casiez, Roussel e Vogel (2012) propõem o **filtro 1€ (One Euro)**, que adapta a suavização

à velocidade do sinal, reduzindo a tremedeira sem atrasar movimentos rápidos; o filtro foi implementado na função OneEuro() de hands.js exatamente com essa finalidade.

8.8 Gráficos 3D na web

Dirksen (2023) e a documentação oficial do Three.js orientaram a montagem da cena (renderizador, câmera em perspectiva, luzes pontuais, névoa, materiais físicos e InstancedMesh). A documentação da MDN Web Docs foi a referência para a Web Audio API (osciladores, filtros, convolução e agendamento preciso), a Pointer Lock API e a Canvas API.

9 CONSIDERAÇÕES FINAIS

O Projeto ÍRIS evoluiu de um protótipo de mira por gestos para um jogo completo, com narrativa ramificada, três setores, inteligência artificial, chefe em fases, trilha sonora adaptativa, cinco segredos e dois modos de controle. O desenvolvimento mostrou, na prática, conceitos de game design (tétrade elementar, narrativa ambiental, curva de dificuldade) e de programação (padrão de módulo, game loop, máquinas de estado, colisão em grade e síntese de áudio).

Entre os aprendizados, destacam-se a importância de testar cedo e com frequência — vários problemas só apareceram em testes automatizados —, de separar dados (história e mapas) da lógica e de pensar a acessibilidade da entrada, como mostrou a adaptação do controle por gestos. Como trabalhos futuros, o grupo identifica: novos setores, um editor de mapas visual, suporte a controle de videogame e uma versão para celular.

REFERÊNCIAS

AMANATIDES, John; WOO, Andrew. A fast voxel traversal algorithm for ray tracing. In: EUROGRAPHICS, 1987, Amsterdã. **Proceedings** [...]. Amsterdã: Eurographics Association, 1987. p. 3-10.

CASIEZ, Géry; ROUSSEL, Nicolas; VOGEL, Daniel. 1€ filter: a simple speed-based low-pass filter for noisy input in interactive systems. In: CONFERENCE ON HUMAN FACTORS IN COMPUTING SYSTEMS (CHI), 2012, Austin. **Proceedings** [...]. New York: ACM, 2012. p. 2527-2530.

CROCKFORD, Douglas. **JavaScript**: the good parts. Sebastopol: O'Reilly Media, 2008.

CSIKSZENTMIHALYI, Mihaly. **Flow**: the psychology of optimal experience. New York: Harper & Row, 1990.

DIRKSEN, Jos. **Learn Three.js**: program 3D animations and visualizations for the web with JavaScript and WebGL. 4. ed. Birmingham: Packt, 2023.

JENKINS, Henry. Game design as narrative architecture. In: WARDRIP-FRUIN, Noah; HARRIGAN, Pat (org.). **First person**: new media as story, performance, and game. Cambridge: MIT Press, 2004. p. 118-130.

KOSTER, Raph. **A theory of fun for game design**. Scottsdale: Paraglyph Press, 2004.

MDN WEB DOCS. **Web Audio API**. Mozilla, [2026]. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/API/Web_Audio_API. Acesso em: 13 nov. 2026.

MDN WEB DOCS. **Pointer Lock API**. Mozilla, [2026]. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/Pointer_Lock_API. Acesso em: 13 nov. 2026.

NYSTROM, Robert. **Game programming patterns**. [S. l.]: Genever Benning, 2014.

SALEN, Katie; ZIMMERMAN, Eric. **Rules of play**: game design fundamentals. Cambridge: MIT Press, 2004.

SCHELL, Jesse. **The art of game design**: a book of lenses. 3. ed. Boca Raton: CRC Press, 2019.

THREE.JS. **Three.js documentation**. [S. l.], [2026]. Disponível em: https://threejs.org/docs/. Acesso em: 13 nov. 2026.

ZHANG, Fan *et al.* MediaPipe Hands: on-device real-time hand tracking. **arXiv**, [S. l.], 2020. arXiv:2006.10214. Disponível em: https://arxiv.org/abs/2006.10214. Acesso em: 13 nov. 2026.